TimeMax — Line by Line Code Explanation

Every file. Every line. Explained simply.

File 1: App.jsx — The Brain of the App

This is the main file. It holds ALL the state (data) and passes it down to every component. Think of it as the control centre.

Imports

Code	What It Does
`import { useState, useMemo }`	Brings in React hooks. useState stores data. useMemo caches expensive calculations.
`import Box from '@mui/material/Box'`	MUI Box is like a div — a container for layout. Accepts sx prop for styling.
`import Typography`	MUI component for all text — headings, body text, captions.
`import Tabs, Tab`	MUI tab bar at the top. Tabs is the container, Tab is each individual tab.
`import Alert`	MUI component that shows coloured info/error/warning/success banners.
`import Container`	MUI wrapper that centres content and limits max width on large screens.
`import IconButton`	MUI button that holds an icon — used for the dark mode toggle.
`import TextField`	MUI input field — used for date pickers and time picker.
`import { createTheme, ThemeProvider }`	Creates a custom MUI theme (colors, font). ThemeProvider applies it to the whole app.
`import CssBaseline`	Resets browser default CSS so the app looks consistent everywhere.
`import DarkModeIcon, LightModeIcon`	Moon and sun icons from MUI icons package.
`import { motion }`	Framer Motion — the animation library. motion.div is an animated div.
`import FocusPicker etc.`	Importing our own custom components from the components folder.
`import { fetchSchedule }`	Importing the API function from utils/api.js.

fadeUp Animation Object

Code	What It Does
`const fadeUp = { hidden: ...`	Defines an animation. hidden = starting state (invisible, shifted down 24px).
`visible: (i = 0) => ({...})`	visible = ending state (fully visible, normal position). i is a custom delay index.
`delay: i * 0.12`	Each section fades in 0.12 seconds after the previous one — creates a stagger effect.
`duration: 0.5, ease: 'easeOut'`	Animation takes 0.5 seconds and slows down at the end (easeOut).

State Variables

Code	What It Does
`const [taskname, setTaskname] = useState('')`	Stores what the user types in the Task Name input. Starts empty.
`const [deadline, setDeadline] = useState('')`	Stores the selected deadline date. Starts empty.
`const [effort, setEffort] = useState('medium')`	Stores effort level selection. Starts as medium.
`const [energy, setEnergy] = useState('moderate')`	Stores energy level selection. Starts as moderate.
`const [tasks, setTasks] = useState([])`	Stores the full list of added tasks. Starts as empty array.
`const [focusminutes, setFocusminutes] = useState(45)`	Stores selected focus duration. Starts at 45 minutes.
`const [schedule, setSchedule] = useState(null)`	Stores the AI-generated schedule. Starts as null (nothing shown).
`const [loading, setLoading] = useState(false)`	True while waiting for AI response. Used to show loading text on button.
`const [error, setError] = useState(null)`	Stores any error message from API call. Null means no error.
`const [commitments, setCommitments] = useState('')`	Stores the user's typed commitments text. Starts empty.
`const [darkMode, setDarkMode] = useState(false)`	True = dark theme, False = light theme. Starts light.
`const [startTime, setStartTime] = useState(...)`	Stores the schedule start time. Auto-filled with current time.
`const [dateFrom, setDateFrom] = useState(todayStr)`	Planning period start date. Defaults to today.
`const [dateTo, setDateTo] = useState(todayStr)`	Planning period end date. Defaults to today (single day plan).

Theme Creation

Code	What It Does
`const theme = useMemo(() => createTheme({...}), [darkMode])`	Creates MUI theme. useMemo means it only recalculates when darkMode changes.
`mode: darkMode ? 'dark' : 'light'`	Ternary: if darkMode is true use dark palette, otherwise use light.
`primary: { main: '#2563EB' }`	Sets the main blue colour used throughout — buttons, chips, etc.
`background.default/paper`	Sets background colours for dark vs light mode.
`fontFamily: 'Roboto'`	Sets the default font for all MUI typography.

Functions

Code	What It Does
`const addtask = () => {`	Function called when user clicks + Add Task button.
`if (!taskname) return`	If task name is empty, do nothing and exit the function.
`const newtask = { taskname, deadline, effort, energy }`	Creates a new task object with all the form values.
`setTasks([...tasks, newtask])`	Spread operator copies existing tasks array, adds new task at the end.
`setTaskname(''); setDeadline('')`	Clears the form inputs after adding so user can add another task.
`const removetask = (index) => {`	Function to delete a task by its position in the array.
`tasks.filter((_, i) => i !== index)`	filter() keeps all tasks EXCEPT the one at the given index.
`const edittask = (index, updatedTask) => {`	Function to update an existing task with new data.
`const updated = [...tasks]`	Creates a copy of the tasks array — never mutate state directly.
`updated[index] = updatedTask`	Replaces the task at the given index with the updated version.
`const generateschedule = async () => {`	async function — can use await inside it.
`if (loading) return`	Prevents double-clicking the button while already loading.
`setLoading(true)`	Shows 'Generating...' text on the button.
`setError(null); setSchedule(null)`	Clears previous error and schedule before new request.
`const result = await fetchSchedule(...)`	Waits for the API call to complete before continuing.
`setSchedule(result)`	Saves the AI response — triggers Schedule component to render.
`catch (err) { setError(err.message) }`	If anything goes wrong, saves the error message to show to user.

`finally { setLoading(false) }`	Always runs — removes loading state whether success or failure.

Return — What Gets Rendered

Code	What It Does
`<ThemeProvider theme={theme}>`	Wraps everything — applies our custom theme to all MUI components inside.
`<CssBaseline />`	Resets browser default styles. Must be inside ThemeProvider.
`<Box sx={{ minHeight: '100vh' ...}}`	Outer wrapper. minHeight 100vh means at least full screen height.
`'&::before': { content: '""'...}`	CSS pseudo-element that adds the noise texture overlay behind everything.
`backgroundImage: url('data:image/svg+xml...')`	SVG noise pattern encoded as a data URL — no separate image file needed.
`opacity: darkMode ? 0.04 : 0.025`	Noise is slightly more visible in dark mode.
`pointerEvents: 'none'`	The noise overlay doesn't block clicks — it's purely visual.
`<Container maxWidth='md'>`	Centres content and limits width to medium (960px). Responsive.
`sx={{ py: { xs: 3, md: 6 } }}`	Responsive padding: 3 units on mobile (xs), 6 units on desktop (md).
`variants={fadeUp} initial='hidden' animate='visible' custom={0}`	Applies fadeUp animation. custom={0} is the delay index — 0 = first to appear.
`fontSize: { xs: '28px', md: '40px' }`	Responsive font size: smaller on mobile, larger on desktop.
`onClick={() => setDarkMode(!darkMode)}`	Arrow function: when clicked, flips darkMode between true and false.
`now.toISOString().split('T')[0]`	Gets today's date in YYYY-MM-DD format for the date input.
`String(now.getHours()).padStart(2, '0')`	Gets current hour, pads to 2 digits (e.g. 9 becomes 09).
`tasks.length > 0 && (<button...>)`	&& means: only render the button if there's at least one task.
`component='button'`	Tells MUI Box to render as a <button> HTML element instead of a div.
`bgcolor: loading ? '#93c5fd' : '#2563eb'`	Light blue when loading, normal blue otherwise.
`<Schedule schedule={schedule} />`	Passes the schedule data down as a prop. Nothing shows if schedule is null.

File 2: utils/api.js — Talking to Gemini AI

This file handles the entire communication with Google's Gemini AI. It builds the prompt, sends it, and returns the parsed result.

Code	What It Does
`const APIKEY = import.meta.env.VITE_GEMINI_API_KEY`	Reads the API key from the .env file. Never hardcode keys in code!
`const URL = `https://generativelanguage...` `	The Gemini API endpoint. The key is appended as a query parameter.
`export const fetchSchedule = async (...) => {`	Exported async function — can be imported and called from App.jsx.
`if (!APIKEY) throw new Error('...')`	If key is missing, throw an error immediately. Stops the function.
`const now = new Date()`	Gets the current date and time object.
`now.toLocaleDateString('en-IN', {...})`	Formats date as 'Monday, 15 March 2026' — the AI reads this.
`const tasklist = tasks.map(t => {...}).join('\n')`	Converts tasks array into a formatted text string, one task per line.
`Math.ceil((new Date(t.deadline) - new Date()) / 86400000)`	Calculates days left: subtracts today from deadline in milliseconds, divides by ms per day.
`const isMultiDay = dateRange.from !== dateRange.to`	True if the date range spans multiple days.
`const prompt = `...` `	Template literal — backticks allow multiline strings with ${variables} inside.
`Return ONLY raw JSON...`	Telling the AI exactly what format to respond in so we can parse it.
`await fetch(URL, { method: 'POST'...})`	Sends HTTP POST request to Gemini API with our prompt in the body.
`JSON.stringify({ contents: [{ parts: [{ text: prompt }] }] })`	Wraps prompt in the exact format Gemini API expects.
`if (!response.ok) throw new Error(...)`	If API returned an error status, throw it so catch() in App.jsx handles it.
`data?.candidates?.[0]?.content?.parts?.[0]?.text`	Optional chaining — safely navigates nested object. Returns undefined if any part is missing.
`text.replace(/```json/g, '').replace(/```/g, '').trim()`	Removes any markdown code fences the AI might add around the JSON.
`return JSON.parse(cleaned)`	Converts the JSON string into a real JavaScript object that React can use.

File 3: TaskForm.jsx — Adding Tasks

Renders the form for adding a new task. Receives all form values and setters as props from App.jsx.

Code	What It Does
`function TaskForm({ taskname, setTaskname, ... })`	Destructures props directly in the function parameter — cleaner than writing props.taskname.
`const daysLeft = deadline ? Math.ceil(...) : null`	Ternary: if deadline exists calculate days left, otherwise return null.
`(new Date(deadline) - new Date()) / 86400000`	Date subtraction gives milliseconds. Divide by 86400000 (ms in a day) to get days.
`Math.ceil(...)`	Rounds UP — so 1.2 days becomes 2 days (generous, not cutting it short).
`deadlineColor = daysLeft <= 2 ? 'error'...`	Chained ternary: red if <= 2 days, orange if <= 5, green otherwise.
`deadlineLabel = daysLeft < 0 ? 'Overdue!'...`	Different label text for each urgency level.
`<Typography variant='caption' color='text.secondary'>`	Small grey label text above each input — tells user what the field is for.
`<TextField placeholder='e.g. Write essay'>`	MUI input field. placeholder shows hint text when empty.
`onChange={(e) => setTaskname(e.target.value)}`	Every keystroke updates the state. e.target.value is what the user typed.
`<TextField select>`	Makes TextField act as a dropdown select instead of a text input.
`<MenuItem value='low'>`	Each option inside the dropdown. value is what gets stored, text is what shows.
`<motion.div initial={{ opacity: 0, y: -6 }}...>`	Deadline badge animates in when it first appears — fades up from slightly below.
`<Chip label={deadlineLabel} color={deadlineColor}>`	MUI Chip is a small rounded badge. Color changes based on urgency.
`<motion.div whileTap={{ scale: 0.95 }}>`	Button shrinks slightly when clicked — gives satisfying press feedback.

File 4: TaskList.jsx — Displaying and Editing Tasks

Shows all added tasks as cards. Each card has view mode and edit mode. Uses Framer Motion for slide-in animations.

Code	What It Does
`const [editingIndex, setEditingIndex] = useState(null)`	Tracks which task is being edited. null = no task in edit mode.
`const [editData, setEditData] = useState({})`	Temporary storage for the edited values before saving.

`const startEdit = (index) => {`	Called when Edit button clicked. Sets the editing index and copies task data.
`setEditData({ ...tasks[index] })`	Spread copies the task object so editing doesn't affect original until saved.
`const saveEdit = () => {`	Called when Save button clicked. Calls edittask prop to update App.jsx state.
`setEditingIndex(null); setEditData({})`	Clears edit mode and temp data after saving.
`const cancelEdit = () => {`	Discards edits and returns to view mode without saving.
`getDaysLeft, getDeadlineLabel, getDeadlineColor`	Helper functions — same deadline logic as TaskForm, repeated here for the task cards.
`const effortEmoji = { low: '🟢', medium: '🟡', high: '🔴' }`	Object mapping effort values to emojis. Used as effortEmoji[task.effort].
`<AnimatePresence>`	Framer Motion wrapper needed for exit animations — without it, exit animations don't run.
`initial={{ opacity: 0, x: -40 }}`	Cards start invisible and shifted 40px to the left.
`animate={{ opacity: 1, x: 0 }}`	Cards animate to fully visible and normal position.
`exit={{ opacity: 0, x: 40 }}`	When removed, cards fade out shifting to the right.
`editingIndex === index ? (...) : (...)`	Ternary: if this card is being edited show edit form, otherwise show view mode.
`setEditData({ ...editData, taskname: e.target.value })`	Spread copies existing editData, then overwrites just the taskname field.
`<CheckIcon />, <CloseIcon />`	MUI icons for save and cancel buttons inside the edit form.
`<motion.div whileTap={{ scale: 0.97 }}>`	Build schedule button subtly shrinks on press.

File 5: FocusPicker.jsx — Choosing Focus Duration

Simple component with preset buttons (15/25/45/90 min) and a custom number input.

Code	What It Does
`function FocusPicker({ focusminutes, setFocusminutes })`	Receives the current value and setter as props from App.jsx.
`{[15, 25, 45, 90].map(mins => (`	Creates an inline array and maps over it — renders one button per value.
`variant={focusminutes === mins ? 'contained' : 'outlined'}`	Selected button is filled (contained), others are just outlined.
`onClick={() => setFocusminutes(mins)}`	Clicking a button updates the state in App.jsx with that duration.

`whileTap={{ scale: 0.93 }}`	Buttons shrink to 93% size when pressed — tactile feedback.
`whileHover={{ scale: 1.05 }}`	Buttons grow slightly on hover — interactive feel.
`transition={{ type: 'spring', stiffness: 300 }}`	Spring physics makes the scale animation feel bouncy and natural.
`borderRadius: '999px'`	Large border radius makes buttons pill-shaped.
`<TextField type='number' label='Custom'>`	Allows typing any custom duration. value controlled by state.
`onChange={(e) => setFocusminutes(Number(e.target.value))}`	Number() converts the string from input into an actual number.

File 6: CommitmentsForm.jsx — Describing Your Week

A textarea where users describe their commitments conversationally. The AI reads this when building the schedule.

Code	What It Does
`function CommitmentsForm({ commitments, setCommitments })`	Receives text value and setter as props.
`<motion.div initial={{ opacity: 0, y: 16 }} animate={{ opacity: 1, y: 0 }}>`	Whole component fades up when it appears.
`border: '1px solid', borderColor: 'divider'`	Uses MUI's divider colour — adapts automatically to dark/light mode.
`borderRadius: '12px'`	Rounded corners on the card container.
`bgcolor: 'action.hover'`	MUI colour token — light grey in light mode, slightly lighter in dark mode.
`fontStyle: 'italic'`	The example text is italicised to distinguish it from UI labels.
`<TextField multiline rows={5} fullWidth>`	Tall textarea. multiline = multiple lines, rows=5 = default height, fullWidth = 100% width.
`availability?.trim() && (<Alert>)`	Optional chaining + &&: only show success alert if text has been entered.
`<Alert severity='success'>`	Green success banner from MUI. Appears after user types something.
`initial={{ opacity: 0, y: -8 }} animate={{ opacity: 1, y: 0 }}`	Success alert slides down from slightly above when it appears.

File 7: Schedule.jsx — Displaying the AI Schedule

Renders the day-by-day schedule returned by the AI. Each day has its own section, each block is a card.

Code	What It Does
`if (!schedule) return null`	If no schedule yet (null), render nothing. Component only shows after AI responds.
`schedule.riskSummary && (<Alert severity='info'>)`	&& means only render if riskSummary exists. Shows blue info banner.
`schedule.warnings?.length > 0 && (`	Optional chaining: only show warnings section if there are warnings.
`schedule.days?.map((day, dayIndex) => (`	Loops over each day in the schedule array.
`initial={{ opacity: 0, y: 30 }} transition={{ delay: dayIndex * 0.1 }}`	Each day fades in with a delay — day 1 first, then day 2, etc.
`bgcolor: '#2563eb', color: 'white'`	Blue pill badge showing the day name (e.g. Monday, 15 March).
`day.schedule?.map((block, i) => (`	Loops over each time block within the day.
`delay: dayIndex * 0.1 + i * 0.07`	Stagger within stagger — each block in each day has a slight delay.
`block.task.toLowerCase().includes('break')`	Checks if task name contains 'break' — break blocks get grey styling.
`borderLeft: '4px solid #2563eb'`	Blue left border on task blocks — grey on break blocks.
`<Chip label={block.time} color='primary' variant='outlined'>`	Time shown as outlined blue chip badge.
`block.tip && (<> <Divider /> <Typography>{block.tip}</Typography> </>)`	Only shows tip and divider if AI included a tip for this block.
`<>...</>`	React Fragment — groups elements without adding an extra div to the DOM.

Every line of your code has a purpose. Now you know what each one does!

TimeMax — timemax-timetable.vercel.app